

Development Plan

Software Engineering

Team 23, Verity
Calvin Salsali
Raymond Tao
Jaden Moore
Richard Han

September 28, 2026

Table 1: Revision History

Date	Developer(s)	Change
2026-09-28	Team 23	Initial development plan, proof of concept plan, and team charter
2026-09-28	Team 23	Revised against course checklist, Avenue rubric, and engineering tools rubric

This development plan records the team’s current decisions for how Verity will be organized, built, reviewed, evaluated, and demonstrated. The plan is intended for team members and future contributors, so decisions are stated concretely enough to guide work while remaining changeable when evidence from the proof of concept, requirements work, or later testing justifies a better approach.

The project repository is github.com/richy1337/Verity.

1 Confidential Information?

Verity is not currently based on confidential industry information and the team does not require proprietary customer data. Development and evaluation will use public, appropriately licensed, synthetic, or team created material unless course staff explicitly approve another source. If confidential information is introduced later, the team will document the governing agreement before using that information in the repository or product.

2 IP to Protect

The project is not currently subject to a separate intellectual property agreement. Team authored code and documentation are intended to be publicly reviewable. Third party models, datasets, libraries, and services remain subject to their own licences and terms, and those constraints will be recorded before the associated dependency is adopted.

3 Copyright License

The project uses the MIT License for team authored source code unless a later dependency or course approved constraint requires a change. The licence is stored in the repository’s `LICENSE` file. This choice supports open review, portfolio use, and future experimentation while keeping licensing obligations straightforward.

4 Team Meeting Plan

The team will hold one scheduled project meeting each week, normally 45 to 60 minutes, with an additional technical working session when an active milestone requires one. At least some meetings will be in person. The team will use available capstone lecture or tutorial time when appropriate rather than publishing a physical meeting location online.

The meeting chair will rotate. Before each scheduled meeting, the chair will open or update a GitHub issue containing the agenda. The meeting will cover current milestone status, completed work, blockers, open design decisions, issue

assignment, and the next concrete commitments. Action items will be written into GitHub issues before the meeting ends.

The project does not currently depend on an external industry supervisor. Meetings with the course TA, instructor, or another approved advisor will be scheduled when required. Questions that materially affect scope, evaluation, or architecture will be documented so that advice can be traced to the decision it influenced.

5 Team Communication Plan

GitHub is the durable source of truth for project work. Issues track tasks, bugs, research questions, meeting actions, and documentation work. Pull requests are used for reviewable code and document changes. Important decisions that would otherwise be lost in chat are summarized in an issue, pull request, or meeting note.

A team group chat may be used for quick coordination, availability, and urgent questions, but work is not considered assigned until it is represented in the repository. Email is used when communication with course staff or an external contact requires a formal record.

6 Team Member Roles

The team will use overlapping roles rather than rigid silos. The assignments below are the proposed starting point for team review; responsibilities may shift as evidence and workload change.

Calvin Salsali

Primary: Frontend and product integration, result presentation, and interaction design. **Secondary:** ML evaluation review, documentation quality assurance, and integration support.

Raymond Tao

Primary: Detection and evaluation, benchmark design, model investigation, and failure analysis. **Secondary:** Backend review, experiment reproducibility, and documentation review.

Jaden Moore

Primary: Backend and system integration, analysis API, and CI and deployment concerns. **Secondary:** Frontend review, testing infrastructure, and model integration support.

Richard Han

Primary: Testing and evaluation infrastructure, reproducibility and data workflow, and privacy and reliability review. **Secondary:** Backend support, model integration, and quality assurance.

Project level responsibilities rotate each milestone. One member is meeting chair, one is issue curator and notetaker, and one is quality assurance reviewer. The quality assurance reviewer checks the relevant course checklist and Avenue rubric before a deliverable is considered complete. For code changes, the author may not be the only reviewer.

Roles are intentionally flexible. At the start of each major milestone, the team will review workload, risk, and learning needs. A member may change primary areas or pair with another member when the current assignment creates a bottleneck or when another person needs domain experience. No technical area is owned by only one person for the entire course.

7 Workflow Plan

7.1 Git and Pull Requests

The default branch is `main`. Work that is more than a trivial correction is completed on short lived branches such as `feature/...`, `fix/...`, or `docs/...`. Pull requests reference the issue they address, explain the change, and identify how the change was verified. Before merge, relevant automated checks must pass and at least one other team member must review substantive changes. Squash merging is preferred when it creates a clearer history.

A pull request is considered reviewable when it has a focused purpose, no unrelated changes, a clear verification note, and enough context that another team member can evaluate the decision without a private explanation. High impact decisions involving evaluation methodology, privacy, or architecture include a short rationale in the pull request or linked issue.

7.2 Issue Management

Issues are small enough for one person to complete in a reasonable work period and contain a clear completion condition. The team uses labels such as `documentation`, `frontend`, `backend`, `ml`, `testing`, `bug`, `meeting`, `research`, `blocked`, and `course-feedback`. Larger work is represented by a milestone or parent issue with smaller implementation issues beneath it.

Priority is recorded as P0, P1, or P2. P0 means the issue blocks a required milestone or creates a serious correctness or safety problem. P1 means required project work that should be completed in the active milestone. P2 means useful work that can move if higher risk tasks need time.

When an issue is closed, the closing comment or linked pull request states why it is complete. Meeting issues and course feedback issues are retained so important decisions remain traceable. GitHub milestones are used for major course deliverables, and a GitHub Project board may be used when parallel implementation work becomes large enough that a board improves visibility.

7.3 CI and CD

The repository already contains a GitHub Actions workflow that compiles LaTeX documentation and publishes project documentation. The team will keep this build green. As implementation begins, CI will also run formatters, linters, unit tests, and focused model or API tests that are suitable for automated execution. Expensive model evaluations will be separated from fast pull request checks so routine development does not wait on large experiments.

Deployment automation will be introduced after the product has a stable vertical slice. A deployment is not considered successful unless the same revision has passed its required checks. If a CI check is repeatedly flaky, the team will fix, quarantine with an issue and owner, or remove the unreliable check rather than normalize ignoring failures.

8 Project Decomposition and Scheduling

GitHub issues and milestones will track the following project level phases. Exact deadlines remain governed by the official course calendar.

Table 2: Initial Project Schedule

Period	Primary focus
Late September	Problem statement, proof of concept plan, development plan, team charter, repository setup
October	Requirements, hazards, benchmark and dataset investigation, evaluation harness, candidate detector experiments
November	Focused proof of concept implementation and live demonstration of the highest risk technical assumption
December to January	Architecture and detailed design, integration plan, first end to end vertical slice
January to February	Core implementation, testing, usability work, model evaluation, reliability improvements
Revision 0 milestone	Demonstrate an integrated working system, record limitations, prioritize remaining gaps
Final term	Verification and validation, final documentation, final demonstration, EXPO preparation

The repository at github.com/richy1337/Verity is the authoritative project location. The team will create or link a GitHub Project board once the initial issue set is populated enough for a board to provide value beyond milestones and labels.

9 Proof of Concept Demonstration Plan

9.1 Primary Implementation Risk

The main implementation risk is not whether the team can build a web interface. The central risk is whether Verity can produce useful and defensible trust signals from real digital content without creating unacceptable false confidence. AI generated and manipulated content changes rapidly, individual detectors can fail on data unlike their training data, and a high aggregate accuracy can hide a harmful false positive rate. If the team cannot measure these limitations and communicate them honestly, the product concept is not viable in its current form.

9.2 Proof of Concept Question

The proof of concept will answer:

Can the team build a reproducible analysis pipeline for a deliberately limited set of supported content types that performs meaningfully better than a naive baseline on held out examples while exposing uncertainty, false positives, and known limitations?

9.3 Demonstration Scope

The proof of concept is intentionally narrower than a product prototype. It tests the highest implementation risk and does not require a polished account system, production database, browser extension, or complete user experience.

The team will:

1. Select a clearly licensed or team created evaluation corpus containing authentic and suspicious or generated examples. The initial intent is to investigate text and images, but the team will narrow to one modality if early evidence shows that supporting both would weaken the experiment.
2. Implement at least one candidate detector or signal path for each content type retained in scope. The team will first evaluate pretrained or existing techniques rather than train a foundation model from scratch.
3. Build a reproducible evaluation harness that records predictions and calculates precision, recall, balanced F1 score, confusion matrices, and false positive rate on held out examples.
4. Demonstrate live running code on previously unseen held out items. The demo will show detector output, uncertainty, supporting signals, and aggregate evaluation results.
5. Include known errors and uncertain cases so the demonstration tests feasibility rather than only showing favourable examples.

The planned demonstration device is a team laptop using a modern browser with the analysis service running locally or in the team’s approved development environment. The team will practice the demonstration before the POC milestone and will assign one presenter to the user flow, one to the evaluation results, and one to limitations and next steps. The exact presenter names will be confirmed with the final team role assignments.

A successful proof of concept will show that at least one supported content analysis path provides signal meaningfully better than a naive baseline, that another team member can reproduce the evaluation from the documented process, and that the output can be surfaced without hiding uncertainty. The initial metrics in the Problem Statement provide direction, but one missed threshold will not automatically invalidate the project if the evidence supports a narrower defensible scope.

9.4 Consequences of a Failed Proof of Concept

A failed POC means the demonstrated evidence does not sufficiently address the primary implementation risk. Examples include a detector performing near the naive baseline, a false positive rate that remains too high for the intended use, results that cannot be reproduced, or confidence values that provide no useful indication of uncertainty.

If this occurs, the team will not mask the result by presenting the detector as more certain than it is. The team will bring the evidence to the TA or instructor and use it to decide whether to narrow the supported modality, change the product from direct authenticity classification to evidence aggregation and verification guidance, replace the candidate technique, or redefine the scope in another course approved way. The team will not commit to a detailed redesign until the failed assumption is understood, since the purpose of the POC is to inform that decision.

10 Engineering Tool Evaluation and Selection

The team will evaluate tools based on whether they reduce project risk and support reproducible engineering, not because they are common choices. The table records the current rationale, important limitations, and how the team plans to adapt each tool to Verity.

Table 3: Engineering Tool Evaluation

Tool	Why it fits Verity	Limitations and planned adaptation
Git and GitHub	Provides version control, pull request review, issue traceability, milestones, and a public project history in one system.	GitHub activity can overvalue commit count and chat decisions can disappear. Contribution is therefore assessed using issues, reviews, tests, documentation, research, and code, while important decisions are copied into durable repository records.
LaTeX, Make, GitHub Actions	The course templates are already LaTeX based and the repository already contains an automated documentation build. This gives reproducible PDFs and lets documentation changes be reviewed like code.	LaTeX has a higher editing cost than collaborative word processors and build failures can be cryptic. The team will keep document changes small, rely on the existing automated build, and use the course templates instead of custom document tooling.
Python and FastAPI	Python has strong machine learning and evaluation libraries. FastAPI provides a lightweight typed service boundary between analysis logic and the product interface.	A separate Python service adds deployment and interface complexity. The team will keep analysis contracts small and typed, and will not introduce multiple services unless the POC shows a clear need.
React and TypeScript	Fits an interactive browser experience and TypeScript improves interface checking between the user interface and analysis API.	Frontend frameworks can add unnecessary complexity. The team will favour a small component structure and will not adopt state management or UI libraries until a concrete need appears.
PyTorch and model libraries	Allow the team to evaluate existing models and detectors quickly instead of spending the capstone training a large model from scratch.	A pretrained model can be easy to run but unreliable on Verity’s intended data. No model will be selected only because it is popular or reports a strong external score. Candidates must be evaluated using the team’s own held out benchmark and failure analysis.

Tool	Why it fits Verity	Limitations and planned adaptation
pytest and a TypeScript test runner	Support fast automated unit and integration tests in the two main language environments.	Test count and coverage can create false confidence. The team will prioritize tests tied to requirements, known failure modes, API contracts, and regressions, using coverage only as a diagnostic signal.
Ruff, Black, ESLint, Prettier	Automate style and basic static checks so review time is spent on design and correctness instead of formatting.	Automated linters do not evaluate architecture or semantic correctness. Passing lint is therefore necessary for code quality but never sufficient for merge approval.

Tool choices will be revisited after the POC. A tool may be replaced if its limitations create measurable project risk or if another option provides a simpler path to the documented requirements.

11 Expected Technology

The initial implementation plan is consistent with the tool evaluation above. The project expects TypeScript with React for the browser interface, Python with FastAPI for the analysis service, and established machine learning libraries such as PyTorch and Hugging Face when their licences permit use. The team will prefer pretrained models or existing detection techniques during the POC. Training or fine tuning will be introduced only when evaluation evidence shows it is justified.

Persistence will remain minimal. A relational database may later support explicit features such as saved analyses, but the standard analysis path should be testable without retaining raw user submissions. Python tests will use `pytest`; TypeScript tests will use a suitable framework such as Vitest or Jest. Coverage tooling may be used diagnostically, but project acceptance will depend on meaningful tests and evaluation results rather than a percentage alone.

Git, GitHub, GitHub Issues, pull requests, milestones, GitHub Actions, and the repository's documentation build are part of the development environment from the beginning.

12 Use of AI and LLMs

The team may use AI assisted development tools for brainstorming, explaining unfamiliar APIs, generating small draft implementations, reviewing code, or

improving documentation. AI generated material will not be merged solely because it appears plausible. Team members remain responsible for understanding code they submit, checking generated claims against primary sources, running tests, and reviewing diffs.

If an LLM is used inside Verity itself, it will not be treated as an authoritative truth source. Its output will be evaluated as one signal or explanation mechanism, with known limitations documented and with test cases designed to reveal confident but unsupported conclusions.

For technical decisions, the team will prefer official documentation, credible evaluation sources, reproducible experiments, and direct tests over uncited model output.

13 Coding Standard

The project will use language appropriate automated standards rather than rely on manual formatting preferences.

- TypeScript code will use strict typing where practical, consistent formatting, linting, clear component boundaries, and descriptive names.
- Python code will follow PEP 8 conventions, use type annotations for public interfaces where practical, and be automatically formatted and linted.
- Functions and modules should have one clear responsibility. Complex model evaluation logic should be separated from HTTP and user interface code.
- Public interfaces and nonobvious algorithms should be documented. Comments should explain why a decision exists rather than restate the code.
- New behaviour should include tests at the lowest useful level. Bug fixes should include a regression test when practical.
- Pull requests should be small enough to review meaningfully and should not mix unrelated refactoring with feature work without a clear reason.

Appendix: Generative AI Use Disclosure

AI Tools Used

ChatGPT (GPT-5.6 Sol) was used as an AI assisted drafting and review tool for the Problem Statement and Development Plan.

How and Where Used

ChatGPT was used to inspect the current Verity repository structure and the course supplied templates and checklists, organize the team's project concept into the required sections, draft and revise prose, test the documents against the Avenue rubric supplied by the team, identify missing rubric items, and propose a clearer proof of concept and engineering tool evaluation. AI assistance materially affected the problem framing, goal wording, workflow plan, POC plan, tool evaluation, expected technology, coding standard, team charter, and proposed reflection text.

ChatGPT was not used as evidence that a detector is accurate, that a benchmark is appropriate, or that a proposed technology has already been implemented. Statements about future tools and metrics are explicitly presented as planned decisions or initial targets.

Verification of AI Generated Content

The AI assisted draft was checked against the current Problem Statement and Development Plan source files, the exact problem statement, development plan, and POC checklist files in the repository, and the Problem Statement, Goals and Development Plan Avenue rubric provided by the team. Repository specific claims were verified directly against the current Verity repository, including the repository URL, MIT licence file, existing GitHub Actions documentation workflow, template structure, and course checklist files.

The team must perform the final human verification before submission. In particular, every member must confirm that the roster, assigned roles, meeting practices, reflection text, custom document choices, POC scope, and technical choices accurately represent team agreements. Proposed model performance targets will be treated as hypotheses to evaluate, not as factual claims about the final system. Any statement the team cannot verify or agree to will be edited or removed before submission.

Appendix: Reflection

1. Why is it important to create a development plan prior to starting the project?

The development plan forces decisions that are easy to postpone when a project is still an idea. For Verity, the most useful outcome was not selecting a specific framework. It was identifying how the team will make work visible, how changes will be reviewed, what the POC must prove, and what evidence would justify changing scope. Without those decisions, the team could spend weeks building an interface while the main risk, whether the analysis can be evaluated and communicated responsibly, remains unresolved. The plan also gives future team discussions a baseline. We can change a decision, but we should be able to explain what evidence made the new choice better than the previous one.

2. What are the advantages and disadvantages of using CI and CD?

The main advantage of CI is fast shared feedback. A documentation build, formatter, linter, or test that runs on every pull request reduces the chance that one member's local setup becomes the only place where the project works. It also makes review more objective because basic checks are automatic. The disadvantage is that CI can create noise if checks are slow, flaky, or disconnected from real project risks. For Verity this matters especially for model evaluation, where a large benchmark should not block every small pull request. Our plan separates quick deterministic checks from expensive evaluations and treats repeated flaky checks as defects to fix rather than warnings to ignore. CD can later reduce manual deployment mistakes, but automating deployment too early would add work before the product has a stable vertical slice.

3. What disagreements did your group have in this deliverable, if any, and how did you resolve them?

No specific disagreement among team members is documented in the material available for this draft. The substantive question raised during drafting is how broadly to define Verity at the start. The concept could expand from generated content to fraud, manipulated images, misinformation, and other trust problems, but that breadth would make the proof of concept and success criteria hard to define. The current plan keeps the user problem broad enough to motivate the product while limiting the initial technical scope to text and images, with additional content types as stretch work. This records the scope choice without attributing an unverified disagreement to the team.

Appendix: Team Charter

External Goals

The team's external goals are to produce a capstone project that is credible enough to demonstrate at the Capstone EXPO, useful enough to show to future employers or potential users, and technically deep enough that each member finishes the year with stronger experience in machine learning systems, software design, testing, and product decision making. The team also aims to maintain a repository and documentation set that another engineer could understand without relying on private team knowledge.

Attendance

Expectations

Team members are expected to attend scheduled meetings on time and remain until agreed action items are assigned. A delay of more than 5 minutes should be communicated when possible. Missing more than two scheduled meetings in a four week period without prior notice triggers a direct team check in.

Acceptable Excuse

Acceptable reasons for missing a meeting or deadline include illness, family or personal emergencies, unavoidable academic conflicts, interviews or recruiting obligations communicated in advance, and other circumstances that a reasonable teammate could not simply reschedule. Forgetting a meeting, repeatedly overcommitting without warning the team, or silently missing an agreed deadline is not acceptable behaviour.

In Case of Emergency

A member facing an emergency should notify the group as soon as practical and identify any issue or deliverable that may be affected. The remaining members will reassign urgent work when necessary. When the member returns, the team will review the schedule and rebalance later work so a short term emergency does not permanently transfer that person's responsibilities to others.

Accountability and Teamwork

Quality

Members should arrive at meetings having completed the preparation they committed to, or with a clear explanation of what blocked them. Code is not considered complete until it is reviewable, tested at an appropriate level, and integrated without breaking required checks. Documentation should be reviewed against the relevant course checklist and rubric at least 24 hours before a planned submission whenever the schedule allows.

Attitude

The team critiques ideas and work products rather than people. Members are expected to ask questions early, share incomplete work when feedback would help, and respond constructively to review comments. Disagreements are first handled directly between the people involved. If that does not resolve the issue, the team discusses it together using project goals and available evidence rather than seniority or volume of opinion. Repeated interpersonal issues may be escalated to the TA. The repository's code of conduct applies to team interactions.

Stay on Track

Each member should have visible current work in GitHub issues during an active project iteration. An issue more than 7 days overdue with no status update, or two consecutive weekly cycles with no substantive contribution and no communicated reason, triggers a team check in. The first response is to identify the blocker, reduce or reassign scope, and agree on a recovery date. If the same pattern repeats, the team documents the problem and meets with the TA before it threatens a graded milestone.

Commit count alone is not used as a contribution measure. The team considers code, review, documentation, testing, research, design, meeting work, and other evidence recorded in the repository. If a member repeatedly fails to meet agreed recovery dates without an acceptable reason, the concern is documented and raised with the TA rather than being silently absorbed by the rest of the team.

When the team reaches a milestone at least 24 hours before the internal deadline with required checks passing, the remaining time is used for review, testing, or helping teammates rather than adding unplanned scope unless the team agrees that the new work is low risk.

Team Building

The team will make space for informal conversation before or after selected meetings and will acknowledge completed milestones rather than treating every meeting only as a status report. At major milestones, the team will briefly discuss what worked, what was frustrating, and one process change worth carrying forward.

Decision Making

The preferred decision method is consensus informed by evidence. For ordinary reversible decisions, if discussion does not reach consensus within a reasonable amount of time, the team uses a majority vote and records the decision. High impact decisions involving project scope, architecture, privacy, or evaluation methodology include a short written rationale. If the team reaches a true deadlock on a high impact decision, it asks the TA, instructor, or appropriate advisor for guidance rather than allow the disagreement to stall the project.